Garbage Recognizer: Image Classification via Deep Convolutional Neural Networks

Raul Spanò

May 19, 2026

Abstract

This project implements an image classification system for waste recognition (Garbage Recognizer) using Convolutional Neural Networks (CNNs). The primary objective is to ensure the scalability of the methodological approach on large datasets, executing a proper model selection phase, and evaluating the generalization capability of the resulting architecture.

Declaration of originality

I declare that this material, which I now submit for assessment, is entirely my/our own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me/us or any other person for assessment on this or any other course of study.

The chosen dataset

The dataset chosen for this project is the Garbage Classification v2, published on Kaggle under the MIT license. The original dataset comprises over 12,000 images divided into 10 macro-categories of waste (e.g., battery, biological, cardboard, glass, metal, plastic, etc.). Prior to use, an automated data cleaning process was performed to identify and remove corrupted files or unsupported formats, ensuring training stability. All 10 available classes were considered to maximize the complexity and realism of the classification task.

Data organization and scalability

As explicitly requested, the data organization was designed to scale efficiently with increasing dataset sizes. Instead of loading the entire image set into the system's RAM, the tf.keras.utils.image_dataset_from_directory and tf.data TensorFlow APIs were utilized. The images were dynamically split into three subsets: a training set (80%), a validation set (10%) used for early stopping and model selection, and a completely unseen test set

(10%). The test set was deliberately introduced to perform a final and unbiased evaluation of the model's true generalization capabilities, ensuring that the final metrics are not influenced by the hyperparameter tuning process. The loading process is performed through batching (batches of 32 images) and prefetching (leveraging the AUTOTUNE feature), allowing the algorithm to theoretically process Terabyte-sized datasets with a constant and highly reduced RAM footprint.

Applied pre-processing techniques

To facilitate network convergence and prevent overfitting, the following pre-processing techniques were applied directly within the model's computational graph:

- Rescaling: Normalization of pixel values to the [0, 1] range by dividing by 255.
- Data augmentation: Stochastic application of random rotations (20%), zoom (20%), and mirroring (horizontal and vertical flips). This forces the model to learn features that are invariant to the object's position and orientation.

Considered algorithms and implementations

The chosen algorithm is a Deep Convolutional Neural Network (CNN) implemented in Python 3 using the Keras/TensorFlow framework. The feature extraction architecture consists of three consecutive convolutional blocks, each containing a Conv2D layer (with ReLU activation) followed by a MaxPooling2D layer for spatial downsampling. The final classifier (fully connected) consists of a Dense layer with 128 neurons, followed by a dropout (0.5) layer to regularize the weights and improve generalization. The final output utilizes a Softmax activation function across 10 nodes to return the multiclass probability distribution.

Experiments and model selection

The experiments were conducted in the Google Colab environment using a T4 GPU hardware accelerator. For the model selection phase, it was decided to evaluate the impact of the learning rate hyperparameter on the Adam optimizer.

- Model 1: Learning rate set to 0.001.
- Model 2: Learning rate set to 0.0001.

For both experiments, an early stopping mechanism based on the validation loss (patience = 3 epochs) was configured to dynamically halt the training as soon as the network began to lose its generalization capability (overfitting), automatically restoring the optimal weights.

Comments and discussion on the experimental results

The empirical results declared Model 1 (LR=0.001) as the most effective. The model completed the scheduled 10 epochs showing a healthy learning trend: both training accuracy

and validation accuracy grew consistently, reaching over 53% accuracy on the validation set. Considering the presence of 10 classes (where a random guess yields a 10% baseline), the result indicates a strong feature extraction capability. It is interesting to note how, in certain phases, the validation metrics outperformed the training ones: this is a classic and positive effect induced by the high dropout rate (0.5) and the data augmentation, which are only active during the training phase.

Model 2 (LR=0.0001), on the other hand, exhibited an excessively slow learning rate. The validation loss stopped decreasing almost immediately, triggering the early stopping mechanism by the third epoch and confirming that, for this specific architecture, a larger optimization step is strictly necessary to escape local minima.

Following the model selection phase, a final evaluation was conducted on the previously isolated test set. Evaluating the models on completely unseen data guarantees an unbiased estimate of their performance in real-world scenarios. Model 1 achieved a test accuracy of approximately 55.45% and a test loss of 1.5794, while Model 2 halted at a test accuracy of 24.58% with a test loss of 1.8385. These results definitively confirm that Model 1 generalizes significantly better, successfully learning the underlying patterns of the dataset without merely memorizing the training samples.

Code availability

The complete source code for this project, including the data pipeline, model training, and evaluation scripts, is publicly available. The code is implemented as a Jupyter notebook designed to be executed on Google Colab to leverage GPU acceleration.

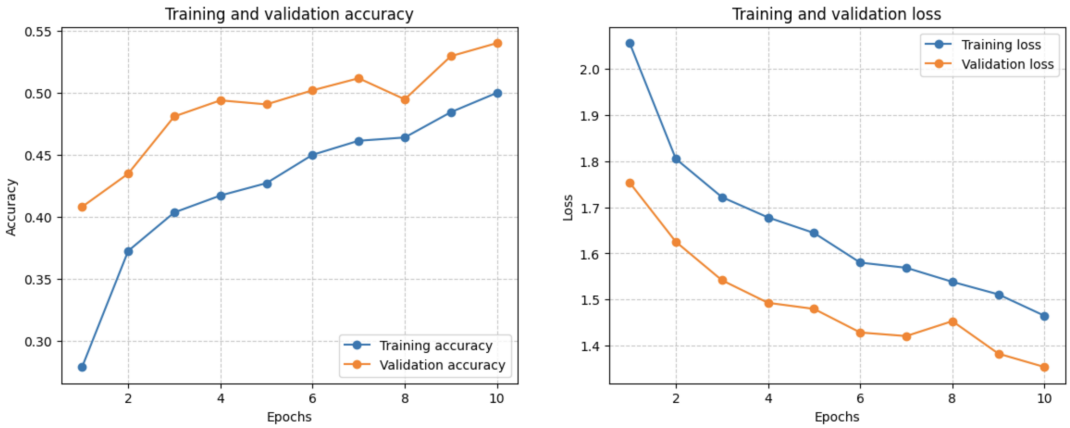

Figure 1: Training and validation accuracy/loss trends for the optimal model (LR=0.001).

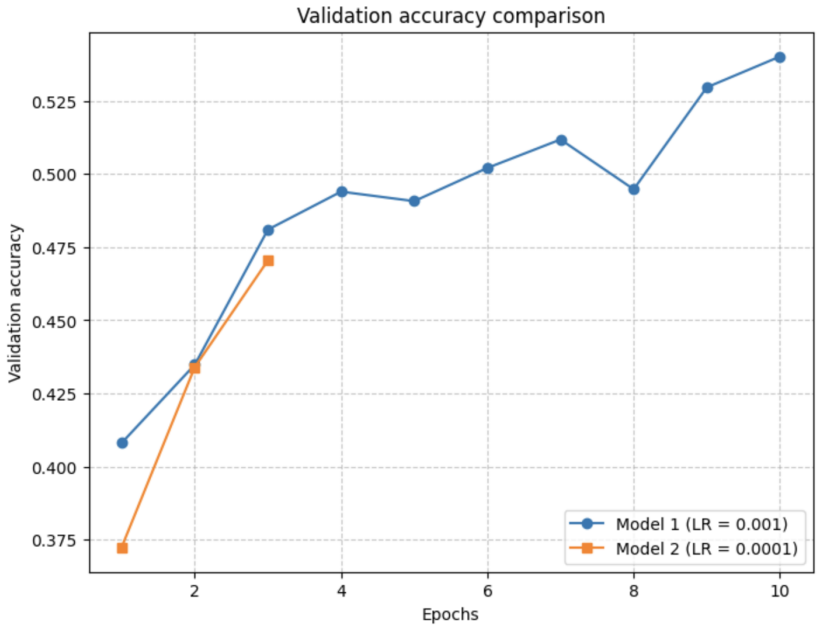

Figure 2: Model Selection: Comparison of Validation Accuracy between different Learning Rates (LR=0.001 vs LR=0.0001).

```
Evaluating the TEST SET
Model 1 (LR=0.001)   -> Test Accuracy: 52.55% | Test Loss: 1.4108
Model 2 (LR=0.0001)  -> Test Accuracy: 35.61% | Test Loss: 1.8559
```

Figure 3: Result of the final evaluation on the Test set (unseen data).